Nome: Bruno Corrêa Dias
Turma: 5a Fase A

# Atividade Arquitetura de Software N2

**1:** O domínio de aplicação do projeto **GameVault** foca-se numa plataforma social e de gestão para jogadores (gamers). O sistema permite o registo e autenticação de utilizadores, a personalização de perfis públicos (incluindo biografia e avatar) e o acompanhamento de métricas de rede social, tais como o número de seguidores, utilizadores seguidos e a quantidade de jogos na biblioteca do utilizador.

**2: Justificativas de Negócio:** A inclusão de um serviço de mensageria no GameVault justifica-se pela necessidade de escalabilidade e desacoplamento de tarefas em background. Por exemplo, quando um utilizador se regista, o envio de um e-mail de boas-vindas ou a sincronização de dados de perfil com um serviço externo de estatísticas de *gamers* (métricas de seguidores/jogos) não deve bloquear a resposta HTTP. A mensageria garante que a API principal responde instantaneamente ao utilizador (melhorando a UX), enquanto o evento UserRegisteredEvent é processado de forma assíncrona."

**Escolhas Técnicas:** "Foi escolhido o **RabbitMQ** (com o protocolo AMQP) pela sua fiabilidade na entrega de mensagens entre os módulos do sistema e facilidade de integração com o ecossistema Node.js. Servirá como *Message Broker*, possuindo *exchanges* e *queues* dedicadas para eventos de sistema."

**3:** A inspeção foi realizada através de uma análise estática manual do código-fonte do backend (Node.js/Express). Como o paradigma predominante no projeto é funcional e modular (não estritamente orientado a objetos), utilizou-se a estratégia recomendada: os ficheiros lógicos (Services, Controllers) foram equiparados a classes, e as funções exportadas a métodos. A análise procurou identificar funções que agregam múltiplos eixos de mudança (violações de SRP), dependências diretas de bibliotecas de infraestrutura em camadas de negócio (violações de DIP) e módulos não abertos para extensão (violações de OCP).

**1. Princípio Violado: SRP (Princípio da Responsabilidade Única)**

- **Ficheiro:** `modules/user/userService.js`
- **Trecho de Código:**

```
if (oldProfilePicture) {
    const oldProfilePicPath = path.join(__dirname, '../../public/uploads/profiles',
oldProfilePicture);
    fs.unlink(oldProfilePicPath, (err) => { ... });
}
```

- **Sugestão de Refatoração:** O serviço de utilizadores está a lidar com a persistência de dados na Base de Dados e, ao mesmo tempo, com a manipulação direta do *File System* (`fs.unlink`). A refatoração ideal seria extrair a lógica de eliminação de ficheiros para um serviço dedicado à infraestrutura (ex: `StorageService` ou `FileStorageAdapter`), mantendo o `userService` focado apenas nas regras de negócio do utilizador.

**2. Princípio Violado: DIP (Princípio da Inversão de Dependência)**

- **Ficheiro:** `modules/user/userService.js`
- **Trecho de Código:**

```
const User = require('./userModel');

// …

const newUser = await User.create({ ... });
```

- **Sugestão de Refatoração:** O módulo de negócio (`userService`) está fortemente acoplado à implementação concreta do Sequelize (`userModel.js`). Se a tecnologia de base de dados mudar, o serviço terá de ser reescrito. A solução é criar uma abstração, como um `UserRepositoryInterface` (ou um objeto que atue como tal em JS), e injetar esse repositório no serviço, invertendo a dependência.

**3. Princípio Violado: SRP (Princípio da Responsabilidade Única)**

- **Ficheiro:** `app.js`
- **Trecho de Código:**

```
const sequelize = require('./config/database');

sequelize.sync({ alter: true })

  .then(() => console.log('Banco de dados sincronizado!'))
```

- **Sugestão de Refatoração:** O ficheiro `app.js` (a equiparar-se a uma classe de configuração do servidor Web) tem a responsabilidade de configurar os *middlewares* do Express e definir as rotas, mas está também a assumir a responsabilidade de gerir a sincronização da base de dados. Esta lógica de base de dados deveria ser movida para o ficheiro de arranque (`bin/www`) ou para um módulo isolado de `DatabaseBootstrapper`.

**4. Princípio Violado: OCP (Princípio Aberto-Fechado)**

- **Ficheiro:** `modules/search/searchService.js`
- **Trecho de Código:**

```
function globalSearch(query) {

  return { games: [], users: [] };

}
```

- **Sugestão de Refatoração:** O comentário revela explicitamente uma violação de OCP: o ficheiro atual *terá de ser modificado* para suportar a busca na base de dados. Para respeitar o OCP, o sistema deveria estar aberto a extensões mas fechado a modificações. Refatoração: Criar uma abstração genérica de procura e ter implementações separadas (ex: `MockSearchProvider` usado atualmente, e futuramente adicionar um `DatabaseSearchProvider`), injetando a desejada sem apagar ou modificar o código antigo.

**5. Princípio Violado: SRP (Princípio da Responsabilidade Única)**

- **Ficheiro:** `modules/user/userController.js`
- **Trecho de Código:**

  ```
  const token = generateToken({
    id: userData.id,
    username: userData.username,
    isAdmin: userData.isAdmin
  });
  ```

- **Sugestão de Refatoração:** O controlador tem a responsabilidade de receber o pedido HTTP e devolver a resposta HTTP. Ao construir o *payload* do JWT e invocar a geração do token, está a assumir regras de segurança e autenticação. O retorno do token (ou a sua geração) deveria estar encapsulado num caso de uso, como `authService.login(...)`, devolvendo o token já gerado ao controlador.

**6. Princípio Violado: DIP (Princípio da Inversão de Dependência)**

- **Ficheiro:** `modules/user/userController.js`
- **Trecho de Código:**

  ```
  const userService = require('./userService');
  // ...
  const newUser = await userService.registerUser(...);
  ```

- **Sugestão de Refatoração:** Tal como o serviço depende diretamente do ORM, o controlador depende diretamente da implementação concreta do serviço. Não existe uma interface ou injeção de dependência. A solução seria injetar o serviço no controlador (por exemplo, através do construtor ou de uma função de *factory*), permitindo facilmente substituir o `userService` por uma versão *mock* durante a execução de testes unitários.